

Fiche Révision CC2

Algorithmique Avancée - Chapitre 2 : Les Arbres
3e année Cybersécurité - École Supérieure d'Informatique et du Numérique
(ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignant : Prof. M. BAKHOUYA

Date : 24 novembre 2025

Table des matières

1 Arbres Binaires de Recherche (ABR)	3
1.1 Introduction et Propriété Fondamentale	3
1.2 Structure d'un Nœud	3
1.3 Parcours d'un ABR	4
1.4 Opération RECHERCHER	4
1.5 MINIMUM et MAXIMUM	5
1.6 SUCCESSEUR et PRÉDÉCESSEUR	6
1.7 Opération INSERTION	7
1.8 Opération SUPPRESSION	9
1.9 Hauteur et Cardinal	11
1.10 Tableau Récapitulatif ABR	12
2 Tas Binaires (Heap)	12
2.1 Définitions et Propriétés	12
2.2 Hauteur d'un Tas	13
2.3 Opération ENTASSER-MAX	14
2.4 Construction d'un Tas	15
2.5 Insertion dans un Tas	16
2.6 Extraction du Maximum	16
2.7 Tri par Tas (Heapsort)	17
2.8 File de Priorité	18
2.9 Tableau Récapitulatif Tas	20
3 Algorithmes de Tri	20
3.1 Tri par Insertion	20
3.2 Tri Fusion (Merge Sort)	21
3.3 Comparaison des Tris	22
4 Exercices de Travaux Dirigés (AG3.pdf)	22
4.1 Exercice 1 : ABR de Hauteurs Différentes	22
4.2 Exercice 2 : Recherche et Successeur	23
4.3 Exercice 3 : Suppression dans un ABR	23
4.4 Exercice 4 : ENTASSER-MAX	23
4.5 Exercice 5 : Insertion et Extraction dans un Tas	24
4.6 Exercice 6 : Tri par Tas	24
4.7 Exercice 7 : Files de Priorité	24
5 Pièges et Erreurs Courantes	25
5.1 Différences ABR vs Tas	25
5.2 Formules à Connaître PAR CŒUR	25
5.3 Erreur : Oublier la Propriété de Complétude du Tas	26
5.4 Erreur : Mélanger les Complexités	26
5.5 Erreur : Ne Pas Dessiner l'Arbre	26
5.6 Erreur : Confondre Parcours Infixe et Tri	26

6	Checklist Avant le CC2	27
6.1	Formules Essentielles	27
6.2	Opérations à Maîtriser	27
6.3	Points d'Attention Finale	28

1 Arbres Binaires de Recherche (ABR)

1.1 Introduction et Propriété Fondamentale

Contexte

Les arbres de recherche sont des structures de données pouvant supporter un nombre d'opérations d'ensemble dynamique :

- Rechercher, Minimum et Maximum
- Insérer et Supprimer
- Prédécesseur et Successeur

Propriété ABR - À CONNAÎTRE PAR CŒUR

Un Arbre Binaire de Recherche est un arbre binaire où pour chaque nœud x :

- Si y est un nœud du sous-arbre gauche de x : $\text{clé}[y] \leq \text{clé}[x]$
- Si y est un nœud du sous-arbre droit de x : $\text{clé}[x] \leq \text{clé}[y]$

En d'autres termes :

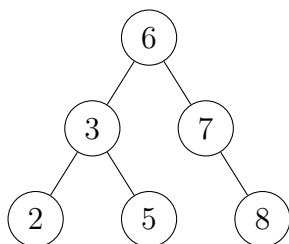
Tous les éléments à GAUCHE $< x$ < Tous les éléments à DROITE

Complexités des Opérations ABR

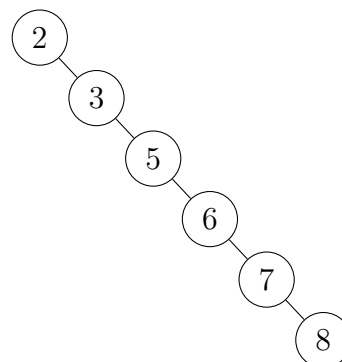
Les opérations basiques sur un ABR dépendent un temps proportionnel à la hauteur h :

- **Arbre complet équilibré** : $\Theta(\log_2 n)$ - Cas optimal
- **Arbre déséquilibré (chaîne)** : $\Theta(n)$ - Pire cas

Équilibré - $h = 2$



Déséquilibré - $h = 4$



1.2 Structure d'un Nœud

```
1 struct noeud_s {  
2     element_t e;           // Valeur du nœud  
3     struct noeud_s *gauche; // Pointeur fils gauche  
4     struct noeud_s *droit;  // Pointeur fils droit
```

```

5     struct noeud_s *parent; // Pointeur parent (optionnel)
6 };
7
8 typedef struct noeud_s* ab_t;

```

Listing 1 – Structure noeud ABR en C

1.3 Parcours d'un ABR

Les 3 Parcours Classiques

- Parcours infixe (Gauche-Racine-Droit) : Imprime les clés **TRIÉES** ✓
- Parcours préfixe (Racine-Gauche-Droit) : Racine en premier
- Parcours postfixe (Gauche-Droit-Racine) : Racine en dernier

```

1 Algorithme: PARCOURS-INFIXE(x)
2 début
3     si x ≠ NIL alors
4         PARCOURS-INFIXE(gauche[x])
5         afficher clé[x]
6         PARCOURS-INFIXE(droit[x])
7     finSi
8 fin

```

Listing 2 – PARCOURS-INFIXE - Pseudo-code (AG3.pdf)

Implémentation C

```

1 void parcours_infixe(ab_t a) {
2     if (a) {
3         parcours_infixe(a->gauche); // Gauche
4         affiche_element(a->e);      // Racine
5         parcours_infixe(a->droit);  // Droit
6     }
7 }

```

Pourquoi infixe donne le tri ?

Grâce à la propriété ABR : on visite d'abord tout ce qui est plus petit (gauche), puis le nœud, puis tout ce qui est plus grand (droit) !

1.4 Opération RECHERCHER

```

1 Algorithme: ARBRE-RECHERCHER(x, k)
2 Entrée: x (pointeur racine), k (clé cherchée)
3 Sortie: Pointeur vers nœud contenant k ou NIL
4
5 début

```

```

6  si x = NIL OU k = clé[x] alors
7      retourner x
8  finSi
9
10 si k < clé[x] alors
11     retourner ARBRE-RECHERCHER(gauche[x], k)
12 sinon
13     retourner ARBRE-RECHERCHER(droit[x], k)
14 finSi
15 fin

```

Listing 3 – ARBRE-RECHERCHER - Pseudo-code (AG3.pdf)

Implémentation C - Version Itérative

```

1  ab_t rechercher(ab_t a, element_t e) {
2      while (a) {
3          int cote = comparer(e, a->e);
4
5          if (cote == 0) {
6              return a; // Element trouve
7          } else if (cote == 1) {
8              a = a->gauche; // Chercher a gauche
9          } else {
10             a = a->droit; // Chercher a droite
11         }
12     }
13
14     return NULL; // Element non trouve
15 }

```

Complexité

$O(h)$ où h = hauteur

Équilibré : $O(\log n)$ | Déséquilibré : $O(n)$

1.5 MINIMUM et MAXIMUM

```

1  Algorithme: ARBRE-MINIMUM(x)
2  début
3      tant que gauche[x] ≠ NIL faire
4          x ← gauche[x]
5      finTantQue
6      retourner x
7  fin
8
9  Algorithme: ARBRE-MAXIMUM(x)
10 début

```

```

11  tant que droit[x] ≠ NIL faire
12      x ← droit[x]
13  finTantQue
14  retourner x
15  fin

```

Listing 4 – ARBRE-MINIMUM - Pseudo-code (AG3.pdf)

Implémentation C

```

1  // Minimum : descendre complètement a gauche
2  ab_t minimum(ab_t a) {
3      if (!a) return NULL;
4
5      while (a->gauche) {
6          a = a->gauche;
7      }
8
9      return a;
10 }
11
12 // Maximum : descendre complètement a droite
13 ab_t maximum(ab_t a) {
14     if (!a) return NULL;
15
16     while (a->droit) {
17         a = a->droit;
18     }
19
20     return a;
21 }

```

Principe

- **Minimum** : Descendre complètement à GAUCHE
- **Maximum** : Descendre complètement à DROITE

Complexité : $O(h)$

1.6 SUCCESSEUR et PRÉDÉCESSEUR

Définition

Le **successeur** d'un nœud x est le nœud possédant la plus petite clé **supérieure** à $\text{clé}[x]$.

```

1  Algorithme: ARBRE-SUCCESSEUR(x)
2  début
3      // Cas 1: Si sous-arbre droit existe
4      si droit[x] ≠ NIL alors

```

```

5     retourner ARBRE-MINIMUM(droit[x])
6 finSi
7
8 // Cas 2: Remonter jusqu'au premier ancêtre "à droite"
9 y ← p[x]
10 tant que y ≠ NIL ET x = droit[y] faire
11     x ← y
12     y ← p[y]
13 finTantQue
14 retourner y
15 fin

```

Listing 5 – ARBRE-SUCCESSEUR - Pseudo-code (AG3.pdf)

Implémentation C

```

1 ab_t successeur(ab_t a) {
2     if (!a) return NULL;
3
4     // Si a un fils droit : successeur = min du sous-arbre droit
5     if (a->droit) {
6         return minimum(a->droit);
7     }
8
9     // Sinon : remonter jusqu'a trouver un parent dont on vient de gauche
10    ab_t parent = a->parent;
11    while (parent && (a == parent->droit)) {
12        a = parent;
13        parent = parent->parent;
14    }
15
16    return parent;
17 }

```

2 Cas pour SUCCESSEUR

Cas 1 : Si le sous-arbre droit n'est pas vide → successeur = MIN du sous-arbre droit

Cas 2 : Si le sous-arbre droit est vide → remonter jusqu'au premier ancêtre dont l'enfant gauche est aussi un ancêtre

1.7 Opération INSERTION

```

1 Algorithme: ARBRE-INSÉRER(T, z)
2 Entrée: T (arbre), z (nouveau nœud avec clé[z] définie)
3
4 début
5     y ← NIL
6     x ← racine[T]

```



```

7
8 // Descendre pour trouver la position
9 tant que x ≠ NIL faire
10     y ← x
11     si clé[z] < clé[x] alors
12         x ← gauche[x]
13     sinon
14         x ← droit[x]
15     finSi
16 finTantQue
17
18 // Attacher z à son parent y
19 p[z] ← y
20
21 si y = NIL alors
22     racine[T] ← z // Arbre était vide
23 sinon si clé[z] < clé[y] alors
24     gauche[y] ← z
25 sinon
26     droit[y] ← z
27 finSi
28 fin

```

Listing 6 – ARBRE-INSÉRER - Pseudo-code (AG3.pdf)

Implémentation C Complète

```

1 void inserer(ab_t *pa, element_t e) {
2     if (!(*pa)) {
3         *pa = nouveau_noeud(e);
4     } else {
5         ab_t abr = *pa;
6         ab_t parent;
7         int cote;
8
9         // Descendre dans l'arbre jusqu'à trouver la position
10        while (abr) {
11            parent = abr;
12            cote = comparer(e, abr->e);
13
14            if (cote == 1) { // e < abr->e
15                abr = abr->gauche;
16            } else if (cote == -1) { // e > abr->e
17                abr = abr->droit;
18            } else { // e == abr->e
19                return; // Element déjà present
20            }
21        }
22
23        // Créer le nouveau noeud
24        abr = nouveau_noeud(e);

```

```

25     abr->parent = parent;
26
27     // Attacher au parent
28     if (cote == 1) {
29         parent->gauche = abr;
30     } else {
31         parent->droit = abr;
32     }
33 }
34 }

```

Principe INSERTION

1. Descendre dans l'arbre comme pour une recherche
2. Conserver le pointeur vers le parent (y)
3. Quand on atteint NIL, insérer le nouveau nœud comme fils de y

Complexité : $O(h)$

1.8 Opération SUPPRESSION

3 Cas de SUPPRESSION

Soit z le nœud à supprimer :

1. **Cas a** : z n'a PAS d'enfant (feuille) $\rightarrow$ supprimer directement
2. **Cas b** : z a UN seul enfant $\rightarrow$ remplacer z par son enfant unique
3. **Cas c** : z a DEUX enfants $\rightarrow$ remplacer z par son successeur, puis supprimer le successeur

```

1  Algorithme: ARBRE-SUPPRIMER(T, z)
2  début
3      // Déterminer le nœud y à détacher
4      si gauche[z] = NIL OU droit[z] = NIL alors
5          y ← z
6      sinon
7          y ← ARBRE-SUCESSEUR(z)
8      finSi
9
10     // x = enfant non-NIL de y (ou NIL)
11     si gauche[y] ≠ NIL alors
12         x ← gauche[y]
13     sinon
14         x ← droit[y]
15     finSi
16
17     // Détacher y de l'arbre
18     si x ≠ NIL alors

```

```

19     p[x] ← p[y]
20 finSi
21
22 si p[y] = NIL alors
23     racine[T] ← x
24 sinon si y = gauche[p[y]] alors
25     gauche[p[y]] ← x
26 sinon
27     droit[p[y]] ← x
28 finSi
29
30 // Si successeur utilisé, copier ses données dans z
31 si y ≠ z alors
32     clé[z] ← clé[y]
33     // Copier données satellites de y vers z
34 finSi
35
36 retourner y
37 fin

```

Listing 7 – ARBRE-SUPPRIMER - Pseudo-code Simplifié (AG3.pdf)

Implémentation C

```

1 void supprimer(ab_t *pa, ab_t y) {
2     if (!y) return;
3
4     ab_t noeud, fils;
5
6     // Si y a 0 ou 1 fils : supprimer y directement
7     if (!(y->gauche) || !(y->droit)) {
8         noeud = y;
9     } else {
10        // Si y a 2 fils : trouver successeur
11        noeud = successeur(y);
12    }
13
14    // Determiner le fils de noeud (peut etre NULL)
15    if (noeud->gauche) {
16        fils = noeud->gauche;
17    } else {
18        fils = noeud->droit;
19    }
20
21    // Attacher fils au parent de noeud
22    if (fils) {
23        fils->parent = noeud->parent;
24    }
25
26    // Mise a jour du parent
27    if (!(noeud->parent)) {

```

```

28     *pa = fils; // noeud etait la racine
29 } else if (noeud == noeud->parent->gauche) {
30     noeud->parent->gauche = fils;
31 } else {
32     noeud->parent->droit = fils;
33 }
34
35 // Si on a utilise le successeur, copier sa valeur dans y
36 if (noeud != y) {
37     y->e = noeud->e;
38 }
39
40 free(noeud);
41 }

```

1.9 Hauteur et Cardinal

Fonctions Auxiliaires en C

```

1 // Hauteur : max des hauteurs des sous-arbres + 1
2 int hauteur(ab_t a) {
3     if (!a) {
4         return -1; // Arbre vide
5     }
6
7     int h_gauche = hauteur(a->gauche);
8     int h_droit = hauteur(a->droit);
9
10    return 1 + max(h_gauche, h_droit);
11 }
12
13 // Cardinal : nombre total de noeuds
14 int cardinal(ab_t a) {
15     if (!a) {
16         return 0;
17     }
18
19    return 1 + cardinal(a->gauche) + cardinal(a->droit);
20 }

```

1.10 Tableau Récapitulatif ABR

Opération	Équilibré	Déséquilibré
RECHERCHER	$O(\log n)$	$O(n)$
MINIMUM/MAXIMUM	$O(\log n)$	$O(n)$
SUCESSEUR/PRÉDÉCESSEUR	$O(\log n)$	$O(n)$
INSÉRER	$O(\log n)$	$O(n)$
SUPPRIMER	$O(\log n)$	$O(n)$
PARCOURS	$O(n)$	$O(n)$
HAUTEUR/CARDINAL	$O(n)$	$O(n)$

TABLE 1 – Toutes les opérations dépendent de la hauteur h

2 Tas Binaires (Heap)

2.1 Définitions et Propriétés

Qu'est-ce qu'un Tas ?

Un tas est une structure de données qui :

- Permet un nouveau type de tri (Tri par Tas)
- Permet l'implémentation de files de priorité
- Peut être vu comme un tableau OU comme un arbre binaire partiellement complet

Représentation d'un Tas (AG3.pdf)

Un tas est représenté par un tableau A avec 2 attributs :

- **longueur** $[A]$: nombre d'éléments du tableau
- **taille** $[A]$: nombre d'éléments du tas rangés dans le tableau

Dans AG3.pdf, la racine est $A[1]$ (indices commencent à 1) :

- **PARENT** (i) : retourner $\lfloor i/2 \rfloor$
- **GAUCHE** (i) : retourner $2i$
- **DROITE** (i) : retourner $2i + 1$

Dans l'implémentation C (indices commencent à 0) :

- **PARENT** (i) : $\lfloor (i - 1)/2 \rfloor$
- **GAUCHE** (i) : $2i + 1$
- **DROITE** (i) : $2i + 2$

ATTENTION - Deux Conventions

Dans AG3.pdf du prof : indices 1..n (comme dans Cormen)

Dans l'implémentation C : indices 0..n-1 (convention programmation)

À l'examen, utiliser la convention du prof (indices 1..n) !

Propriété Tas Max

$$A[\text{PARENT}(i)] \geq A[i]$$

La valeur d'un nœud est au plus égale à celle de son parent.

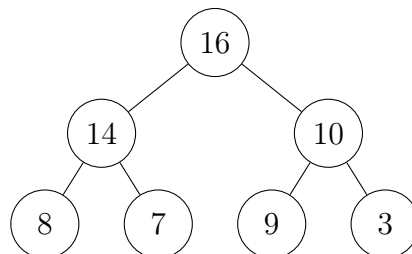
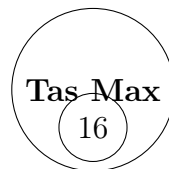
La racine contient la valeur la plus grande.

Propriété Tas Min

$$A[\text{PARENT}(i)] \leq A[i]$$

La valeur d'un nœud est au minimum égale à celle de son parent.

La racine contient la valeur la plus petite.



2.2 Hauteur d'un Tas

Hauteur

Un tas de n nœuds a une hauteur $h = O(\log n)$.

Démonstration (AG3.pdf) :

- Il y a 2^i nœuds au niveau $i = 0, 1, 2, \dots, h - 1$
- Au moins 1 nœud au niveau h
- Donc : $n \geq 1 + 2 + 4 + \dots + 2^{h-1} + 1 = 2^h$
- Par conséquent : $n \geq 2^h \Rightarrow h \leq \log_2 n$

2.3 Opération ENTASSER-MAX

Principe

ENTASSER-MAX fait "descendre" une valeur $A[i]$ dans le tas pour restaurer la propriété de tas max.

Précondition : Les sous-arbres enracinés en $\text{GAUCHE}(i)$ et $\text{DROITE}(i)$ sont déjà des tas max.

```
1  Algorithme: ENTASSER-MAX(A, i)
2  début
3      g ← GAUCHE(i)
4      d ← DROITE(i)
5
6      // Trouver le plus grand parmi A[i], A[g], A[d]
7      si g ≤ taille[A] ET A[g] > A[i] alors
8          plus_grand ← g
9      sinon
10         plus_grand ← i
11     finSi
12
13     si d ≤ taille[A] ET A[d] > A[plus_grand] alors
14         plus_grand ← d
15     finSi
16
17     // Si A[i] n'est pas le plus grand, échanger et continuer
18     si plus_grand ≠ i alors
19         échanger A[i] ↔ A[plus_grand]
20         ENTASSER-MAX(A, plus_grand)
21     finSi
22 fin
```

Listing 8 – ENTASSER-MAX - Pseudo-code (AG3.pdf)

Implémentation C (indices 0..n-1)

```
1  void entasserMax(noeud x, int i) {
2      int gauche = 2*i + 1;
3      int droit = 2*i + 2;
4      int indice_max = i;
5
6      // Comparer avec fils gauche
7      if ((gauche < x->taille) && (x->tab[gauche] > x->tab[indice_max])) {
8          indice_max = gauche;
9      }
10
11     // Comparer avec fils droit
12     if ((droit < x->taille) && (x->tab[droit] > x->tab[indice_max])) {
13         indice_max = droit;
14     }
```

```

15
16 // Si le max n'est pas i, echanger et continuer
17 if (indice_max != i) {
18     echangenoeud(x, i, indice_max);
19     entasserMax(x, indice_max); // Recursif !
20 }
21 }

```

Complexité ENTASSER-MAX

$O(\log n)$

Car on descend au maximum sur la hauteur $h = \log_2 n$.

2.4 Construction d'un Tas

```

1 Algorithme: CONSTRUIRE-TAS-MAX(A)
2 début
3     taille[A] ← longueur[A]
4
5     // Entasser de bas en haut, depuis le dernier nœud interne
6     pour i ← [longueur[A]/2] jusqu'à 1 (décroissant) faire
7         ENTASSER-MAX(A, i)
8     finPour
9 fin

```

Listing 9 – CONSTRUIRE-TAS-MAX - Pseudo-code (AG3.pdf)

Implémentation C - Construction $O(n)$

```

1 void construireTasMax(noeud x) {
2     // Commencer au dernier noeud interne
3     int debut = (x->taille) / 2 - 1;
4
5     // Entasser tous les noeuds internes, de bas en haut
6     for (int i = debut; i >= 0; i--) {
7         entasserMax(x, i);
8     }
9 }

```

Pourquoi commencer à $\lfloor n/2 \rfloor$?

Les éléments $A[\lfloor n/2 \rfloor + 1 \dots n]$ sont les **feuilles** de l'arbre.
Chaque feuille est déjà un tas à 1 élément! On entasse uniquement les nœuds internes.

Complexité CONSTRUIRE-TAS-MAX

$$O(n)$$

PAS $O(n \log n)$!

Analyse amortie (AG3.pdf) :

- Niveau i : 2^i nœuds, chacun fait au plus $h - i$ échanges
- Total : $\sum_{i=0}^h 2^i (h - i) = 2^h \sum_{j=0}^h \frac{j}{2^j} \leq 2^h \cdot 2 = 2n$
- Donc : $O(n)$

2.5 Insertion dans un Tas

Implémentation C - Insertion $O(\log n)$

```
1 void insererTasMax(noeud x, int e) {  
2     // Ajouter l'element en fin de tas  
3     x->tab[x->taille] = e;  
4  
5     int i = x->taille;  
6  
7     // Faire remonter l'element jusqu'a restaurer la propriete  
8     while (i > 0 && x->tab[parent(i)] < x->tab[i]) {  
9         echangenoeud(x, i, parent(i));  
10        i = parent(i);  
11    }  
12  
13    x->taille++;  
14 }
```

Complexité Insertion

$$O(\log n)$$

Car l'élément remonte au maximum jusqu'à la racine (hauteur $\log_2(n)$).

2.6 Extraction du Maximum

```
1 Algorithme: EXTRAIRE-MAX-TAS(A)  
2 début  
3     si taille[A] < 1 alors  
4         erreur "tas vide"  
5     finSi  
6  
7     max ← A[1]  
8     A[1] ← A[taille[A]]  
9     taille[A] ← taille[A] - 1  
10    ENTASSER-MAX(A, 1)
```

```

11     retourner max
12 fin

```

Listing 10 – EXTRAIRE-MAX-TAS - Pseudo-code (AG3.pdf)

Implémentation C

```

1 int extraireMax(noeud x) {
2     if (x->taille == 0) {
3         perror("Erreur : tas vide");
4         exit(EXIT_FAILURE);
5     }
6
7     // Sauvegarder le max (racine)
8     int max = x->tab[0];
9
10    // Remplacer la racine par le dernier element
11    x->taille--;
12    x->tab[0] = x->tab[x->taille];
13
14    // Restaurer la propriete de tas
15    if (x->taille > 0) {
16        entasserMax(x, 0);
17    }
18
19    return max;
20 }

```

2.7 Tri par Tas (Heapsort)

```

1 Algorithme: TRI-PAR-TAS(A)
2 début
3     CONSTRUIRE-TAS-MAX(A) // O(n)
4
5     // Extraire répétitivement le max
6     pour i ← longueur[A] jusqu'à 2 (décroissant) faire
7         échanger A[1] ↔ A[i]
8         taille[A] ← taille[A] - 1
9         ENTASSER-MAX(A, 1)
10    finPour
11 fin

```

Listing 11 – TRI-PAR-TAS - Pseudo-code (AG3.pdf)

Implémentation C Complète

```

1 void triTas(int *tab, int taille) {
2     noeud x = malloc(sizeof(*x));
3     x->tab = tab;

```

```

4   x->taille = taille;
5   x->mem = taille;
6
7   // 1. Construire tas max :  $O(n)$ 
8   construireTasMax(x);
9
10  // 2. Extraire repetitivement le max :  $n$  fois  $O(\log n)$ 
11  for (int i = taille - 1; i > 0; i--) {
12      // Echanger racine (max) avec dernier element
13      echangenoeud(x, 0, i);
14
15      // Reduire la taille du tas
16      x->taille--;
17
18      // Restaurer propriete sur le tas reduit
19      entasserMax(x, 0);
20  }
21
22  free(x);
23 }

```

Complexité TRI-PAR-TAS

$$O(n \log n)$$

- Construction : $O(n)$
- Boucle : $n - 1$ appels à ENTASSER-MAX, chacun $O(\log n)$
- Total : $O(n) + O(n \log n) = O(n \log n)$

Avantages Tri par Tas

- Complexité $O(n \log n)$ dans TOUS les cas
- Tri **sur place** : $O(1)$ mémoire supplémentaire
- Pas de récursion

2.8 File de Priorité

Définition

Une file de priorité gère un ensemble S d'éléments avec des clés définissant les priorités.

Opérations (File-Max) :

- $\text{MAXIMUM}(S)$: retourner l'élément de clé maximale
- $\text{EXTRAIRE-MAX}(S)$: retourner et supprimer l'élément de clé maximale
- $\text{AUGMENTER-CLÉ}(S, x, k)$: augmenter la clé de x à k
- $\text{INSÉRER}(S, x)$: insérer x dans S

```
1 Algorithme: MAXIMUM-TAS(A)
2 début
3   retourner A[1]
4 fin
5 // Complexité:  $\Theta(1)$ 
```

Listing 12 – MAXIMUM-TAS - Pseudo-code (AG3.pdf)

```
1 Algorithme: AUGMENTER-CLÉ-TAS(A, i, clé)
2 début
3   si clé < A[i] alors
4     erreur "nouvelle clé plus petite"
5   finSi
6
7   A[i] ← clé
8
9   // Faire remonter l'élément
10  tant que i > 1 ET A[PARENT(i)] < A[i] faire
11    échanger A[i] ↔ A[PARENT(i)]
12    i ← PARENT(i)
13  finTantQue
14 fin
15 // Complexité:  $O(\log n)$ 
```

Listing 13 – AUGMENTER-CLÉ-TAS - Pseudo-code (AG3.pdf)

```
1 Algorithme: INSÉRER-TAS-MAX(A, clé)
2 début
3   taille[A] ← taille[A] + 1
4   A[taille[A]] ←  $-\infty$ 
5   AUGMENTER-CLÉ-TAS(A, taille[A], clé)
6 fin
7 // Complexité:  $O(\log n)$ 
```

Listing 14 – INSÉRER-TAS-MAX - Pseudo-code (AG3.pdf)

2.9 Tableau Récapitulatif Tas

Opération	Complexité
ENTASSER-MAX	$O(\log n)$
CONSTRUIRE-TAS-MAX	$O(n)$
TRI-PAR-TAS	$O(n \log n)$
MAXIMUM	$O(1)$
EXTRAIRE-MAX	$O(\log n)$
AUGMENTER-CLÉ	$O(\log n)$
INSÉRER	$O(\log n)$

TABLE 2 – Toutes les opérations de file de priorité en $O(\log n)$ sauf MAX en $O(1)$

3 Algorithmes de Tri

3.1 Tri par Insertion

```
1 void tri_insertion(int tab[], int n) {
2     int i, j, cle;
3
4     for (i = 1; i < n; i++) {
5         cle = tab[i];
6         j = i - 1;
7
8         // Deplacer les elements > cle vers la droite
9         while (j >= 0 && tab[j] > cle) {
10             tab[j + 1] = tab[j];
11             j--;
12         }
13
14         // Insérer cle a sa position
15         tab[j + 1] = cle;
16     }
17 }
```

Listing 15 – Tri par Insertion en C

Complexité Tri Insertion

- **Meilleur cas** (tableau déjà trié) : $O(n)$
- **Cas moyen** : $O(n^2)$
- **Pire cas** (tableau trié à l'envers) : $O(n^2)$

$$\text{Nombre de comparaisons pire cas : } \frac{n(n-1)}{2}$$

3.2 Tri Fusion (Merge Sort)

```
1 void tri_fusion(int tab[], int debut, int fin) {
2     // Cas de base : tableau de taille 0 ou 1
3     if (fin - debut <= 1) {
4         return;
5     }
6
7     // Calculer le milieu
8     int milieu = debut + (fin - debut) / 2;
9
10    // Trier recursivement les deux moitiés
11    tri_fusion(tab, debut, milieu); // Gauche
12    tri_fusion(tab, milieu, fin);   // Droit
13
14    // Fusionner les deux moitiés triées
15    int size_gauche = milieu - debut;
16    int size_droite = fin - milieu;
17
18    int *gauche = copier_tableau(tab, debut, milieu);
19    int *droite = copier_tableau(tab, milieu, fin);
20
21    int *resultat = fusionner(gauche, size_gauche, droite, size_droite);
22
23    // Recopier dans le tableau original
24    for (int i = 0; i < (fin - debut); i++) {
25        tab[debut + i] = resultat[i];
26    }
27
28    free(gauche);
29    free(droite);
30    free(resultat);
31 }
```

Listing 16 – Tri Fusion - Diviser pour Régner

Complexité Tri Fusion

- **Tous les cas** : $O(n \log n)$ ✓
- **Espace** : $O(n)$ (nécessite mémoire supplémentaire)
- **Stable** : préserve l'ordre des éléments égaux

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

3.3 Comparaison des Tris

Algorithme	Meilleur	Moyen	Pire	Espace
Tri Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Tri Fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Tri par Tas	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

TABLE 3 – Comparaison des complexités

4 Exercices de Travaux Dirigés (AG3.pdf)

4.1 Exercice 1 : ABR de Hauteurs Différentes

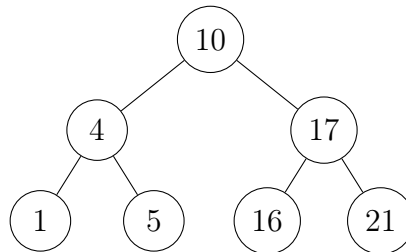
Énoncé

Dessinez les arbres binaires de recherche de hauteur 2, 3, 4, 5 et 6 pour le même ensemble de clés $S = \{1, 4, 5, 10, 16, 17, 21\}$.

Solution :

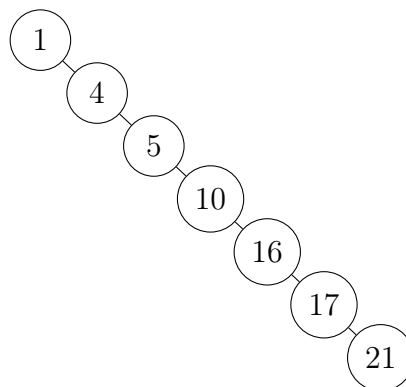
Pour obtenir des ABR de hauteurs différentes avec le même ensemble, il suffit de changer l'ordre d'insertion.

Hauteur 2 (optimal - arbre équilibré) :



Ordre d'insertion : 10, 4, 17, 1, 5, 16, 21

Hauteur 6 (pire cas - liste chaînée) :



Ordre d'insertion : 1, 4, 5, 10, 16, 17, 21 (ordre croissant)

4.2 Exercice 2 : Recherche et Successeur

Énoncé

Soit l'arbre suivant. Donner :

1. Le résultat de parcours infixé
2. Le chemin pour rechercher 27
3. Les successeurs de 32 et 66

Solution :

1. Parcours infixé :

7, 9, 16, 22, 25, 27, 28, 29, 30, 32, 34, 35, 37, 66, 67, 88

2. Chemin pour rechercher 27 :

34 → 22 → 29 → 25 → 27

- 27 < 34 → aller à gauche
- 27 > 22 → aller à droite
- 27 < 29 → aller à gauche
- 27 > 25 → aller à droite
- 27 = 27 → trouvé!

3. Successeurs :

- **Successeur de 32 :** 34 (remonter jusqu'au premier ancêtre dont on vient de gauche)
- **Successeur de 66 :** 67 (minimum du sous-arbre droit n'existe pas, donc remonter)

4.3 Exercice 3 : Suppression dans un ABR

Énoncé

Supprimer les nœuds 51 et 55 de l'arbre donné.

Solution :

Suppression de 51 (a un seul fils à droite) :

Cas b : remplacer 51 par son fils unique (55)

Suppression de 55 (feuille) :

Cas a : supprimer directement

4.4 Exercice 4 : ENTASSER-MAX

Énoncé

Illustrer l'action de ENTASSER-MAX(A,2) avec :

$$A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$$

Solution :

Étape 1 : État initial (indice 2 = valeur 4)

Comparer 4 avec ses fils : $\max(4, 14, 7) = 14 \rightarrow$ échanger 4 et 14

Étape 2 : Après 1er échange

Comparer 4 avec ses fils : $\max(4, 2, 8) = 8 \rightarrow$ échanger 4 et 8

Résultat final :

Tableau final : [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]

4.5 Exercice 5 : Insertion et Extraction dans un Tas

Énoncé

Soit un tas max donné.

1. Insérer un élément de clé 17
2. En repartant du tas initial, extraire le maximum

Solution 1 : Insertion de 17

1. Ajouter 17 en fin (devient une feuille)
2. Faire remonter 17 : comparer avec parent, échanger si nécessaire
3. Répéter jusqu'à restaurer la propriété tas max

Solution 2 : Extraction du maximum

1. Sauvegarder max (racine)
2. Remplacer racine par dernier élément
3. ENTASSER-MAX(A, 1) pour restaurer la propriété

4.6 Exercice 6 : Tri par Tas

Énoncé

Donner le résultat de l'algorithme de tri par tas sur :

[2, 8, 1, 9, 8, 8, 6, 5, 3, 4]

Solution :

Étape 1 : Construction du tas max

Dernier nœud interne = $\lfloor 10/2 \rfloor = 5$

Entasser de 5 à 1 pour obtenir : [9, 8, 8, 5, 8, 1, 6, 2, 3, 4]

Étape 2 : Tri (extraction répétée)

Échanger max avec dernier, réduire taille, entasser, répéter...

Tableau final trié : [1, 2, 3, 4, 5, 6, 8, 8, 8, 9]

4.7 Exercice 7 : Files de Priorité

Énoncé

File de priorité max avec opérations :

- MAXIMUM(A)
- EXTRAIRE-MAX(A)
- INSERER-TAS-MAX(A, x)

Donner les algorithmes et complexités.

Solution : Voir section 2.4 (Files de Priorité)

Complexités :

- MAXIMUM(A) : $O(1)$
- EXTRAIRE-MAX(A) : $O(\log n)$
- INSÉRER(A, x) : $O(\log n)$

5 Pièges et Erreurs Courantes

5.1 Différences ABR vs Tas

Ne PAS confondre !

	ABR	Tas Max
Propriété	Gauche < Racine < Droit	Parent $\geq$ Enfants
Structure	Peut être déséquilibré	TOUJOURS complet
Parcours trié	Infixe donne tri	Aucun parcours ne trie
Recherche	$O(\log n)$ équilibré	$O(n)$ toujours
Max/Min	Descendre gauche/droite	Racine ($O(1)$)

5.2 Formules à Connaître PAR CŒUR

Indices dans AG3.pdf (commencent à 1)

Pour un tas représenté par tableau A (indices 1..n) :

- PARENT(i) = $\lfloor i/2 \rfloor$
- GAUCHE(i) = $2i$
- DROITE(i) = $2i + 1$

Dans l'implémentation C (indices 0..n-1) :

- parent(i) = $\lfloor (i - 1)/2 \rfloor$
- gauche(i) = $2i + 1$
- droit(i) = $2i + 2$

Complexités essentielles :

- ABR équilibré : toutes opérations $O(\log n)$
- ABR déséquilibré : toutes opérations $O(n)$
- ENTASSER : $O(\log n)$
- CONSTRUIRE-TAS : $O(n)$ ← ATTENTION!
- TRI-PAR-TAS : $O(n \log n)$

5.3 Erreur : Oublier la Propriété de Complétude du Tas

Rappel Important

Un tas est TOUJOURS un arbre binaire **complet** :

- Tous les niveaux sont remplis sauf possiblement le dernier
- Le dernier niveau est rempli de gauche à droite

Cela permet la représentation efficace par tableau !

5.4 Erreur : Mélanger les Complexités

Attention aux Contextes

Construction d'un tas :

- Méthode directe (entasser de bas en haut) : $O(n)$ ✓
- Méthode insertion (élément par élément) : $O(n \log n)$

Tri par Tas :

- Construction : $O(n)$
- Extractions : $n \times O(\log n) = O(n \log n)$
- **Total** : $O(n) + O(n \log n) = O(n \log n)$ ✓

5.5 Erreur : Ne Pas Dessiner l'Arbre

Conseil ESSENTIEL

Lors d'un exercice sur ABR ou Tas :

1. **TOUJOURS dessiner l'arbre** au brouillon
2. Vérifier visuellement la propriété
3. Tracer les opérations étape par étape

Le dessin permet d'éviter 90% des erreurs ! ✓

5.6 Erreur : Confondre Parcours Infixe et Tri

Important

- **ABR** : Parcours infixe donne le TRI ✓
- **Tas** : AUCUN parcours ne donne le tri !

Pour trier avec un tas, il faut utiliser le TRI PAR TAS (extractions répétées).

6 Checklist Avant le CC2

6.1 Formules Essentielles

À Savoir PAR CŒUR

ABR :

- Propriété : Gauche < Racine < Droit
- Parcours infixe = tri croissant
- Complexités équilibré : $O(\log n)$
- Complexités déséquilibré : $O(n)$

Tas (indices AG3.pdf : 1..n) :

- $\text{PARENT}(i) = \lfloor i/2 \rfloor$
- $\text{GAUCHE}(i) = 2i$
- $\text{DROITE}(i) = 2i + 1$
- Propriété Tas Max : $\text{PARENT}(i) \geq i$

Complexités Tas :

- Entasser : $O(\log n)$
- Construction directe : $O(n)$
- Insertion/Extraction : $O(\log n)$
- Tri par Tas : $O(n \log n)$

Tris :

- Insertion : $O(n)$ meilleur, $O(n^2)$ pire
- Fusion : $O(n \log n)$ toujours
- Tas : $O(n \log n)$ toujours, sur place

6.2 Opérations à Maîtriser

Savoir Faire Sans Hésiter

Sur ABR :

- ☐ Insérer un élément
- ☐ Rechercher un élément
- ☐ Trouver min/max
- ☐ Trouver successeur/prédécesseur
- ☐ Supprimer un nœud (3 cas)
- ☐ Parcourir (infixe, préfixe, postfixe)
- ☐ Calculer hauteur et cardinal
- ☐ Vérifier propriété ABR

Sur Tas :

- ☐ Entasser (heapify)

- ☐ Construire tas (méthode directe $O(n)$)
- ☐ Insérer élément
- ☐ Extraire max/min
- ☐ Convertir tableau en tas
- ☐ Vérifier propriété tas

Tris :

- ☐ Écrire tri insertion
- ☐ Écrire tri fusion
- ☐ Écrire tri par tas
- ☐ Comparer complexités
- ☐ Choisir tri adapté au contexte

6.3 Points d'Attention Finale

CRITICAL

1. ABR : Vérifier propriété pour TOUT le sous-arbre, pas juste parent-enfant
2. Tas : TOUJOURS complet, hauteur = $O(\log n)$
3. CONSTRUIRE-TAS : $O(n)$, PAS $O(n \log n)$!
4. Indices : À l'examen, utiliser convention AG3.pdf (1..n)
5. Suppression ABR : 3 cas distincts (0, 1, ou 2 fils)
6. SUCCESSEUR : 2 cas (sous-arbre droit existe ou pas)
7. Parcours infixe : Donne tri pour ABR, PAS pour tas

BON COURAGE POUR TON CC2!

Références

- **Cours** : Prof. Mohamed BAKHOUYA, *Algorithmique et Structures de Données Avancées*, Chapitre 2 (AG3.pdf), UIR 2025-2026
- **Ouvrage** : Thomas H. CORMEN et al., *Introduction à l'algorithmique*, 2ème édition, DUNOD
- **TPs** : TP4 (ABR), TP5 (Tas), TP6 (Tri), TP7 (Files de Priorité)
- **Assistant IA** : Claude (Sonnet 4.5), Anthropic, <https://claude.ai>

Remerciements

Ce document de révision a été créé pour faciliter la préparation au CC2 d'Algorithmique Avancée. Il synthétise le Chapitre 2 du cours du Prof. BAKHOUYA et inclut des exemples, exercices corrigés et conseils pratiques.

*Document créé le 24 novembre 2025
Documents autorisés - Imprimez et annotez !*